

Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий
Проектирование баз знаний

Отчёт по лабораторной работе №2
Вариант 11

Выполнил:
Студент гр. 321701

Лукашов В.А.

Проверил:

Ерофеев И.А.

Минск 2025

Содержание

Постановка задачи.....	3
Концептуальное проектирование.....	4
Логическое проектирование.....	8
Физическое проектирование.....	9
Тексты основных запросов, функций, процедур и триггеров.....	13
Интерфейс приложения.....	25
Инструментальные средства, которые использованы при разработке.....	28

Постановка задачи: Словесное описание предметной области

Вариант 11. «Учёт приказов и распоряжений»

Предприятие, имеющее в своей структуре ряд иерархически подчиняющихся подразделений.

Словесное описание предметной области: Необходимо хранить информацию о внутренних приказах, распоряжениях или других подобных документах, сроках их выполнения и исполнителях. Например: название документа - "Указание о введении в действие плана по совершенствованию охраны труда", мероприятие, описываемое в документе – "Установить порядок выпуска приказов по случаю нарушения охраны труда", - исходящий корреспондент (человек, готовивший приказ) – гл. инженер.

Информация о исходящем корреспонденте: название подразделения, должность корреспондента, ФИО. Приказ, распоряжение характеризуются номером, датой, содержанием, мероприятием, ответственным за выполнение (название подразделения, должность, ФИО), датой выполнения мероприятия, пометкой о выполнении мероприятия. В одном документе может быть перечислено несколько мероприятий. Фиксируется дата выхода документа. Распоряжение подписывает сотрудник ранга руководителя предприятия, необходимо фиксировать должность и ФИО.

Необходимо реализовать выполнения следующих функций:

- Добавление/редактирование/удаление информации о внутренних приказах, распоряжениях или других подобных документах.
- Добавление/редактирование/удаление информации о корреспондентах.
- Просмотр списка исходящих корреспондентов, мероприятия по которым на заданную дату не были выполнены – текущая дата, дата выхода документа, название мероприятия, дата выполнения, ФИО, должность, название подразделения ответственного.
- Просмотр списка мероприятий за заданный период – дата начала периода, дата окончания периода, название мероприятия, выполнено или нет;
- Просмотр списка всех зафиксированных документов заданного типа (например, приказов) на текущую дату, отсортированных по дате документа – название предприятия, текущая дата, название вида документа, список документов – номер, дата документа, пометка о выполнении.

Концептуальное проектирование: Диаграммы бизнес-процессов (нотация BPMN) / Диаграммы вариантов использования (ВИ, Use-Case) / Спецификации ВИ

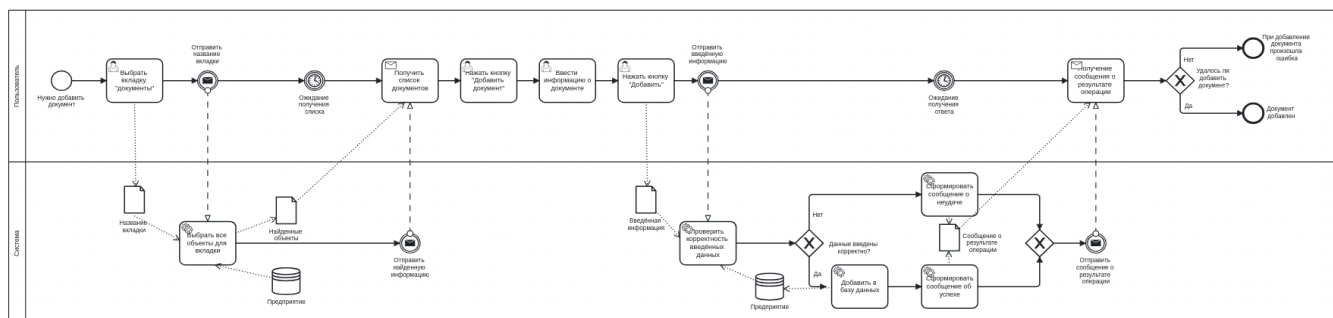


Рисунок 1. BPMN-диаграмма добавления информации о внутренних приказах, распоряжениях или других подобных документах предприятия

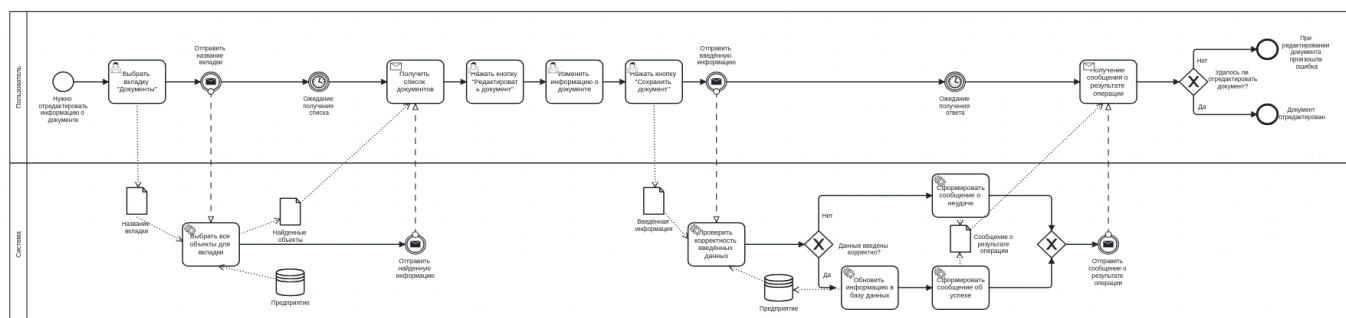


Рисунок 2. BPMN-диаграмма редактирования информации о внутренних приказах, распоряжениях или других подобных документах предприятия

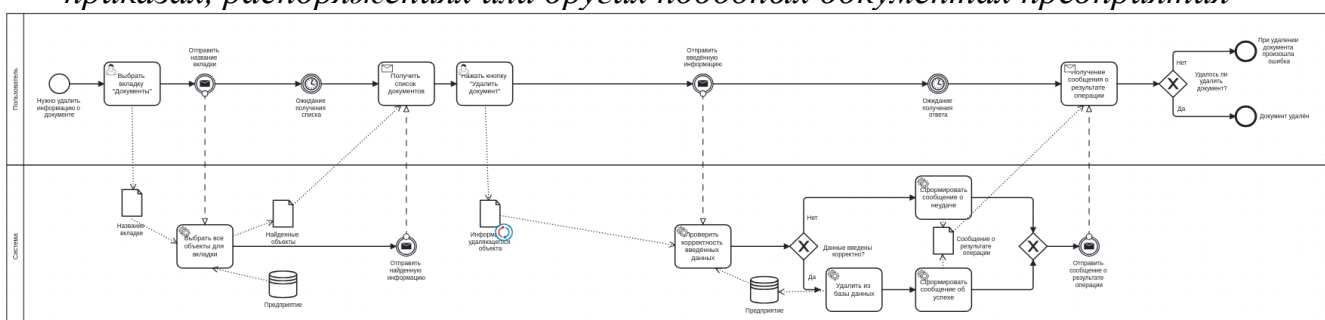


Рисунок 3. BPMN-диаграмма редактирования информации о внутренних приказах, распоряжениях или других подобных документах предприятия

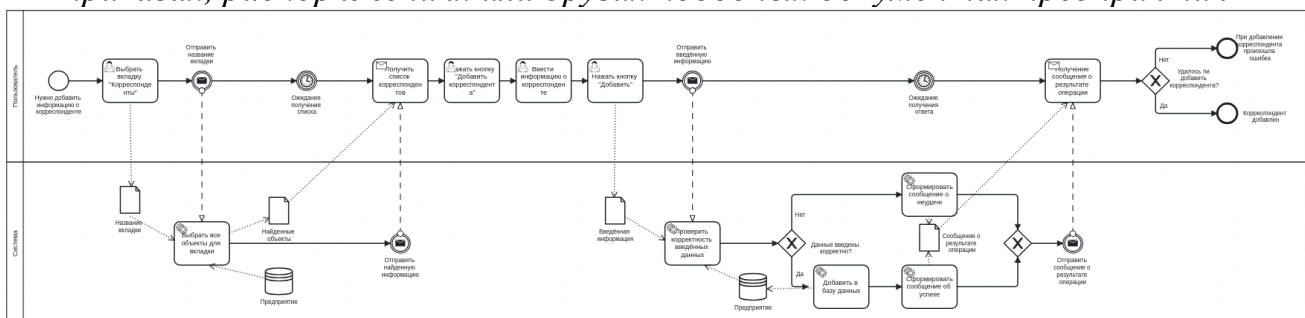


Рисунок 4. BPMN-диаграмма добавления информации о корреспондентах

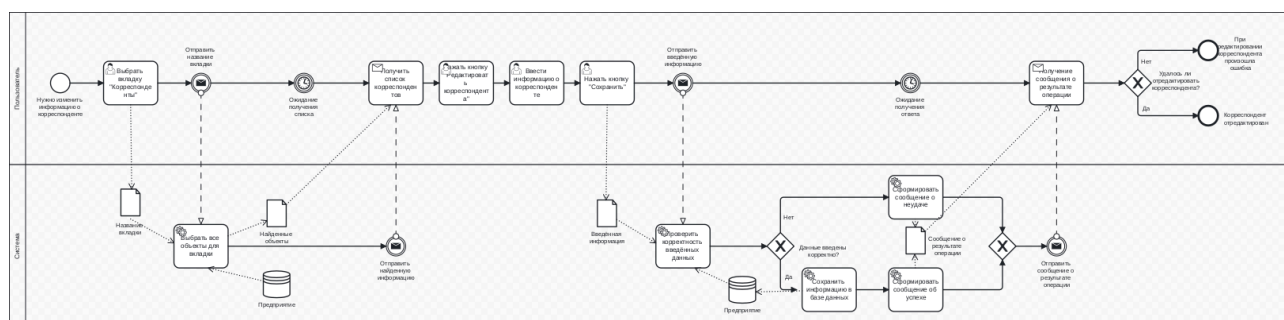


Рисунок 5. BPMN-диаграмма изменения информации о корреспондентах

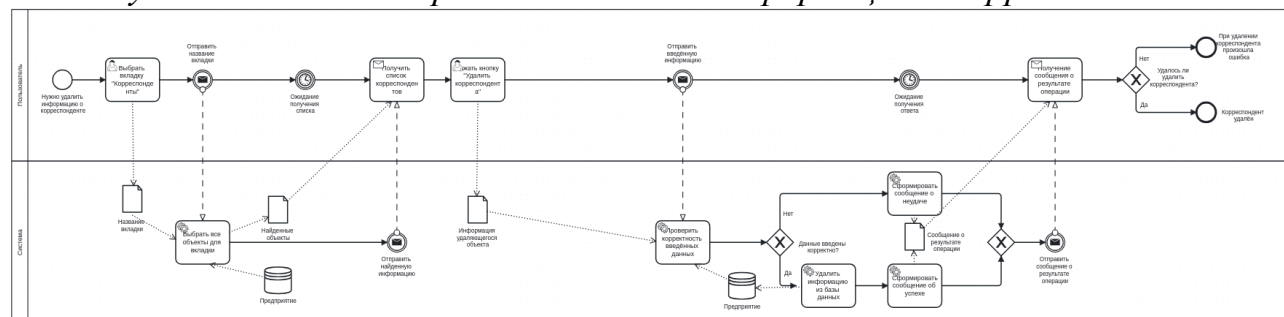


Рисунок 6. BPMN-диаграмма удаления информации о корреспондентах

На представленной диаграмме вариантов использования (Рис. 10) показана концептуальная модель взаимодействия различных групп пользователей системы с информационной системой “Система учёта приказов и распоряжений”.

Поле	Описание
Название Use-Case	Редактирование документа
ID	UC-01
Акторы	Корреспондент
Описание	Данный вариант использования позволяет автору документа редактировать любой реквизит документа и список мероприятий до момента его отправки на подпись.
Расширяется (Extends)	“Добавление мероприятия к документу” “Удаление мероприятия из документа” “Редактирование мероприятия”
Предусловия	1. Пользователь аутентифицирован в системе как Автор. 2. Документ существует. 3. Пользователь является автором данного документа.

Постусловия	Изменения в документе сохранены в базе данных.
Основной сценарий	1. Автор выбирает вкладку "Документы" 2. Автор находит в списке нужный документ. 3. Автор нажимает кнопку "Редактировать" 4. Система предоставляет форму редактирования документа. 5. Автор изменяет необходимые данные (заголовок, содержание, даты). 6. Автор сохраняет изменения. 7. Система проверяет корректность данных и сохраняет изменения в БД.
Альтернативные сценарии	А. Добавление мероприятия: автор добавляет новое мероприятие в документ через соответствующую кнопку в форме редактирования. Б. Отправка на подпись: после сохранения изменений автор может сразу отправить документ на подпись.
Исключения	Ошибка проверки корректности введенных данных: если данные пользователя не корректны, то система выдаст сообщение об ошибке.

Таблица 1. Спецификация ВИ "Редактирование документа"

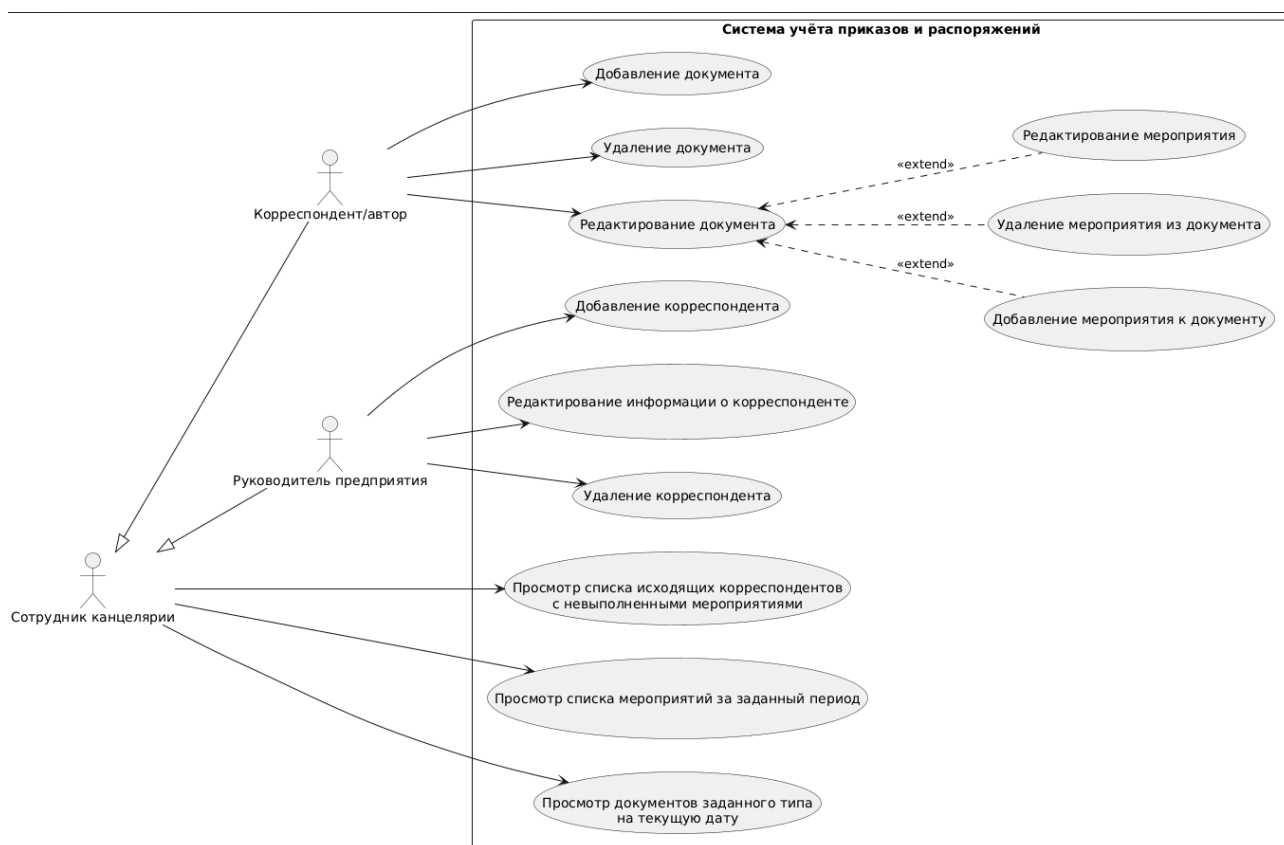


Рисунок 7. Диаграмма вариантов использования системы

Логическое проектирование: Диаграмма сущность-связь (ER-диаграмма)

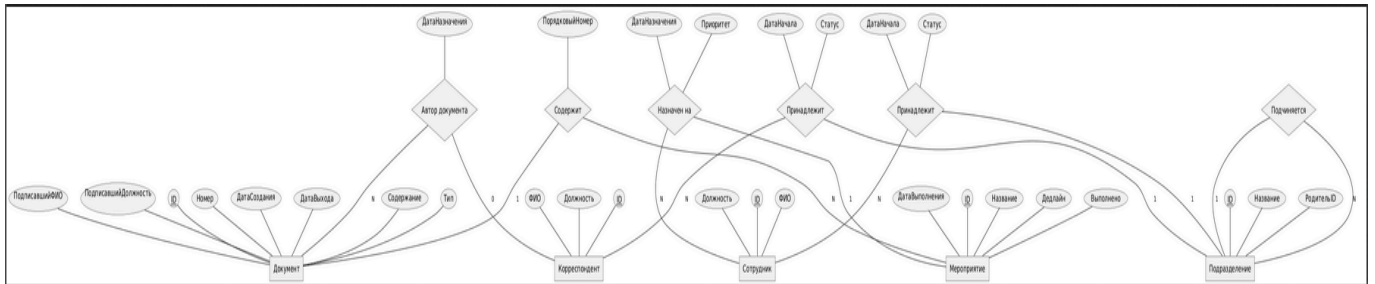


Рисунок 8. Диаграмма сущность-связь (ER-диаграмма) в нотации Чена

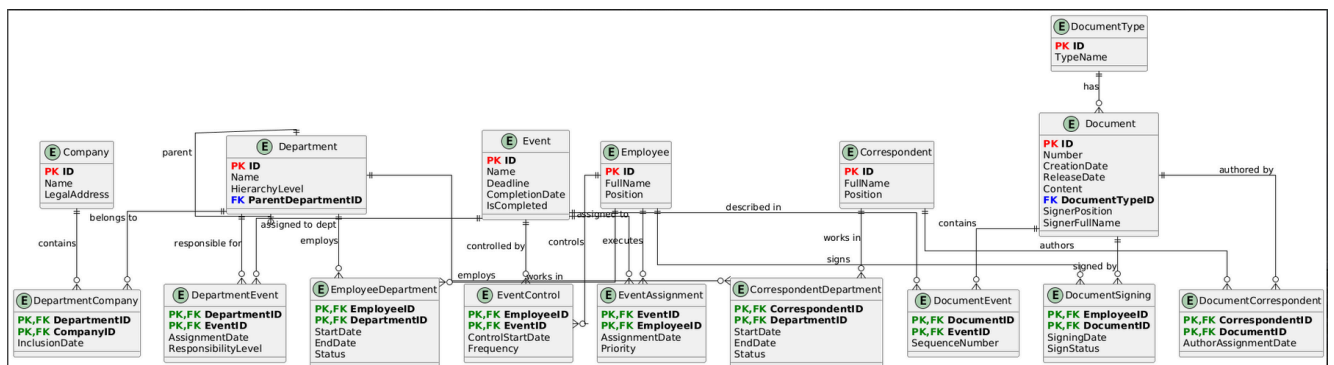


Рисунок 9. Диаграмма сущность-связь (ER-диаграмма) в нотации Мартина

Физическое проектирование: Схема БД

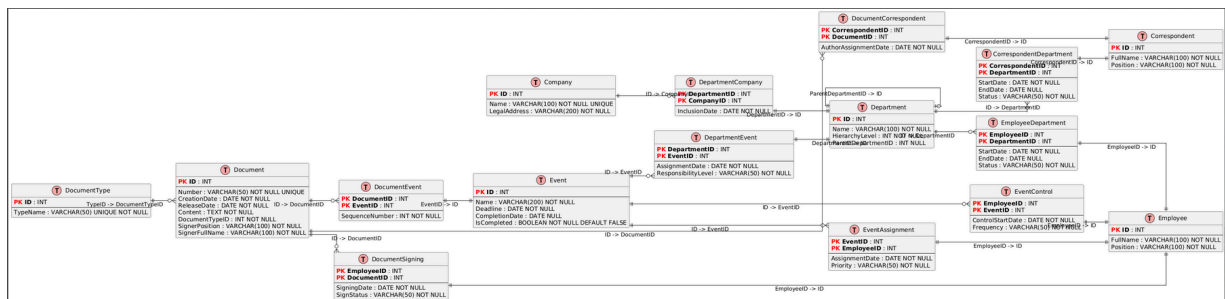


Рисунок 10. Схема базы данных

Скрипт для создания базы данных:

```
CREATE TABLE IF NOT EXISTS DocumentType (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    TypeName VARCHAR(50) UNIQUE NOT NULL
);

INSERT INTO DocumentType (TypeName) VALUES ('Приказ'), ('Распоряжение'), ('Указание')
ON CONFLICT (TypeName) DO NOTHING;

CREATE TABLE IF NOT EXISTS Company (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Name VARCHAR(100) NOT NULL UNIQUE,
    LegalAddress VARCHAR(200) NOT NULL
);

CREATE TABLE IF NOT EXISTS Department (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    HierarchyLevel INT NOT NULL,
    ParentDepartmentID INT NULL,
    FOREIGN KEY (ParentDepartmentID) REFERENCES Department(ID) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS Correspondent (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Position VARCHAR(100) NOT NULL
);

CREATE TABLE IF NOT EXISTS Document (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Number VARCHAR(50) NOT NULL UNIQUE,
    CreatorDate DATE NOT NULL,
    ReleaseDate DATE NOT NULL,
    Content TEXT NOT NULL,
    DocumentTypeID INT NOT NULL,
    SignerFullName VARCHAR(100) NOT NULL,
    FOREIGN KEY (DocumentTypeID) REFERENCES DocumentType(ID) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS DocumentEvent (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    DocumentID INT NOT NULL,
    EventID INT NOT NULL,
    FOREIGN KEY (DocumentID) REFERENCES Document(ID) ON DELETE SET NULL,
    FOREIGN KEY (EventID) REFERENCES Event(ID) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS Event (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Name VARCHAR(200) NOT NULL,
    Deadline DATE NOT NULL,
    CompletionDate DATE NULL,
    IsComplete BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE IF NOT EXISTS DocumentCorrespondent (
    DocumentCorrespondentID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    DocumentID INT NOT NULL,
    CorrespondentID INT NOT NULL,
    AssignmentDate DATE NOT NULL,
    FOREIGN KEY (DocumentID) REFERENCES Document(ID) ON DELETE SET NULL,
    FOREIGN KEY (CorrespondentID) REFERENCES Correspondent(ID) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS EmployeeDepartment (
    EmployeeID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    DepartmentID INT NOT NULL,
    StartDate DATE NOT NULL,
    EndDate DATE NULL,
    Status VARCHAR(50) NOT NULL,
    FOREIGN KEY (EmployeeID) REFERENCES Employee(ID) ON DELETE SET NULL,
    FOREIGN KEY (DepartmentID) REFERENCES Department(ID) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS EventControl (
    EventID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    EmployeeID INT NOT NULL,
    ControlDate DATE NOT NULL,
    Frequency VARCHAR(50) NOT NULL,
    FOREIGN KEY (EventID) REFERENCES Event(ID) ON DELETE SET NULL,
    FOREIGN KEY (EmployeeID) REFERENCES Employee(ID) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS EventAssignment (
    EventID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    AssignmentDate DATE NOT NULL,
    Priority VARCHAR(50) NOT NULL,
    FOREIGN KEY (EventID) REFERENCES Event(ID) ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS Employee (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Position VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS Employee (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Position VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS Document (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Number VARCHAR(50) NOT NULL UNIQUE,
    CreationDate DATE NOT NULL,
    ReleaseDate DATE NOT NULL,
    Content TEXT NOT NULL,
    DocumentTypeID INT NOT NULL, -- Новое поле
    SignerPosition VARCHAR(100) NOT NULL,
    SignerFullName VARCHAR(100) NOT NULL,
    FOREIGN KEY (DocumentTypeID) REFERENCES DocumentType(ID)
);
```

```
CREATE TABLE IF NOT EXISTS Event (
    ID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    Name VARCHAR(200) NOT NULL,
    Deadline DATE NOT NULL,
    CompletionDate DATE NULL,
    IsCompleted BOOLEAN NOT NULL DEFAULT FALSE
);
```

```
CREATE TABLE IF NOT EXISTS DocumentCorrespondent (
    CorrespondentID INT NOT NULL,
    DocumentID INT NOT NULL,
    AuthorAssignmentDate DATE NOT NULL,
    PRIMARY KEY (CorrespondentID, DocumentID),
    FOREIGN KEY (CorrespondentID) REFERENCES Correspondent(ID) ON DELETE CASCADE,
    FOREIGN KEY (DocumentID) REFERENCES Document(ID) ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS DocumentEvent (
    DocumentID INT NOT NULL,
    EventID INT NOT NULL,
    SequenceNumber INT NOT NULL,
    PRIMARY KEY (DocumentID, EventID),
    FOREIGN KEY (DocumentID) REFERENCES Document(ID) ON DELETE CASCADE,
```

FOREIGN KEY (EventID) REFERENCES Event(ID) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS EventAssignment (
EventID INT NOT NULL,
EmployeeID INT NOT NULL,
AssignmentDate DATE NOT NULL,
Priority VARCHAR(50) NOT NULL,
PRIMARY KEY (EventID, EmployeeID),
FOREIGN KEY (EventID) REFERENCES Event(ID) ON DELETE CASCADE,
FOREIGN KEY (EmployeeID) REFERENCES Employee(ID) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS DocumentSigning (
EmployeeID INT NOT NULL,
DocumentID INT NOT NULL,
SigningDate DATE NOT NULL,
SignStatus VARCHAR(50) NOT NULL,
PRIMARY KEY (EmployeeID, DocumentID),
FOREIGN KEY (EmployeeID) REFERENCES Employee(ID) ON DELETE CASCADE,
FOREIGN KEY (DocumentID) REFERENCES Document(ID) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS CorrespondentDepartment (
CorrespondentID INT NOT NULL,
DepartmentID INT NOT NULL,
StartDate DATE NOT NULL,
EndDate DATE NULL,
Status VARCHAR(50) NOT NULL,
PRIMARY KEY (CorrespondentID, DepartmentID),
FOREIGN KEY (CorrespondentID) REFERENCES Correspondent(ID) ON DELETE CASCADE,
FOREIGN KEY (DepartmentID) REFERENCES Department(ID) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS EmployeeDepartment (
EmployeeID INT NOT NULL,
DepartmentID INT NOT NULL,
StartDate DATE NOT NULL,
EndDate DATE NULL,
Status VARCHAR(50) NOT NULL,
PRIMARY KEY (EmployeeID, DepartmentID),
FOREIGN KEY (EmployeeID) REFERENCES Employee(ID) ON DELETE CASCADE,

```
FOREIGN KEY (DepartmentID) REFERENCES Department(ID) ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS DepartmentCompany (
    DepartmentID INT NOT NULL,
    CompanyID INT NOT NULL,
    InclusionDate DATE NOT NULL,
    PRIMARY KEY (DepartmentID, CompanyID),
    FOREIGN KEY (DepartmentID) REFERENCES Department(ID) ON DELETE CASCADE,
    FOREIGN KEY (CompanyID) REFERENCES Company(ID) ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS DepartmentEvent (
    DepartmentID INT NOT NULL,
    EventID INT NOT NULL,
    AssignmentDate DATE NOT NULL,
    ResponsibilityLevel VARCHAR(50) NOT NULL,
    PRIMARY KEY (DepartmentID, EventID),
    FOREIGN KEY (DepartmentID) REFERENCES Department(ID) ON DELETE CASCADE,
    FOREIGN KEY (EventID) REFERENCES Event(ID) ON DELETE CASCADE
);
```

```
CREATE TABLE IF NOT EXISTS EventControl (
    EmployeeID INT NOT NULL,
    EventID INT NOT NULL,
    ControlStartDate DATE NOT NULL,
    Frequency VARCHAR(50) NOT NULL,
    PRIMARY KEY (EmployeeID, EventID),
    FOREIGN KEY (EmployeeID) REFERENCES Employee(ID) ON DELETE CASCADE,
    FOREIGN KEY (EventID) REFERENCES Event(ID) ON DELETE CASCADE
);
```

```
CREATE INDEX IF NOT EXISTS idx_document_creation_date ON Document(CreationDate);
CREATE INDEX IF NOT EXISTS idx_document_type ON Document(DocumentTypeID); --
Изменено
CREATE INDEX IF NOT EXISTS idx_event_deadline ON Event(Deadline);
CREATE INDEX IF NOT EXISTS idx_event_completion_date ON Event(CompletionDate);
CREATE INDEX IF NOT EXISTS idx_event_is_completed ON Event(IsCompleted);
CREATE INDEX IF NOT EXISTS idx_document_event_sequence ON
DocumentEvent(SequenceNumber);
CREATE INDEX IF NOT EXISTS idx_event_assignment_priority ON EventAssignment(Priority);
CREATE INDEX IF NOT EXISTS idx_document_signing_status ON
DocumentSigning(SignStatus);
```

Тексты основных запросов, функций, процедур и триггеров

-- 1. Функция создания документа

```
CREATE OR REPLACE FUNCTION CreateDocument(
    p_number VARCHAR(50),
    p_creation_date DATE,
    p_release_date DATE,
    p_content TEXT,
    p_document_type_id INT, -- Изменено
    p_signer_position VARCHAR(100),
    p_signer_full_name VARCHAR(100)
)
RETURNS INT
LANGUAGE plpgsql
AS
$$
DECLARE
    new_id INT;
BEGIN
    INSERT INTO Document (Number, CreationDate, ReleaseDate, Content, DocumentTypeID,
        SignerPosition, SignerFullName)
        VALUES (p_number, p_creation_date, p_release_date, p_content, p_document_type_id,
            p_signer_position, p_signer_full_name)
        RETURNING ID INTO new_id;

    RETURN new_id;
END;
$$;
```

-- 2. Процедура редактирования документа

```
CREATE OR REPLACE PROCEDURE UpdateDocument(
    p_document_id INT,
    p_number VARCHAR(50),
    p_creation_date DATE,
    p_release_date DATE,
    p_content TEXT,
    p_document_type_id INT,
    p_signer_position VARCHAR(100),
    p_signer_full_name VARCHAR(100)
)
```

```

LANGUAGE plpgsql
AS
$$
BEGIN
    UPDATE Document
    SET Number = p_number,
        CreationDate = p_creation_date,
        ReleaseDate = p_release_date,
        Content = p_content,
        DocumentTypeID = p_document_type_id, -- Изменено
        SignerPosition = p_signer_position,
        SignerFullName = p_signer_full_name
    WHERE ID = p_document_id;
END;
$$;

-- 3. Функция создания мероприятия
CREATE OR REPLACE FUNCTION CreateEvent(
    p_name VARCHAR(200),
    p_deadline DATE,
    p_completion_date DATE DEFAULT NULL,
    p_is_completed BOOLEAN DEFAULT FALSE
)
RETURNS INT
LANGUAGE plpgsql
AS
$$
DECLARE
    new_id INT;
BEGIN
    INSERT INTO Event (Name, Deadline, CompletionDate, IsCompleted)
    VALUES (p_name, p_deadline, p_completion_date, p_is_completed)
    RETURNING ID INTO new_id;

    RETURN new_id;
END;
$$;

-- 4. Процедура редактирования мероприятия
CREATE OR REPLACE PROCEDURE UpdateEvent(
    p_event_id INT,
    p_name VARCHAR(200),
    p_deadline DATE,

```

```

    p_completion_date DATE DEFAULT NULL,
    p_is_completed BOOLEAN DEFAULT FALSE
)
LANGUAGE plpgsql
AS
$$
BEGIN
    UPDATE Event
    SET Name = p_name,
        Deadline = p_deadline,
        CompletionDate = p_completion_date,
        IsCompleted = p_is_completed
    WHERE ID = p_event_id;
END;
$$;

```

-- 5. Процедура добавления связи документа и корреспондента

```

CREATE OR REPLACE PROCEDURE AddDocumentCorrespondent(
    p_correspondent_id INT,
    p_document_id INT,
    p_author_assignment_date DATE
)
LANGUAGE plpgsql
AS
$$
BEGIN
    INSERT INTO DocumentCorrespondent (CorrespondentID, DocumentID,
    AuthorAssignmentDate)
    VALUES (p_correspondent_id, p_document_id, p_author_assignment_date)
    ON CONFLICT (CorrespondentID, DocumentID) DO NOTHING;
END;
$$;

```

-- 6. Процедура добавления мероприятия к документу

```

CREATE OR REPLACE PROCEDURE AddEventToDocument(
    p_document_id INT,
    p_event_id INT,
    p_sequence_number INT
)
LANGUAGE plpgsql
AS
$$
BEGIN

```

```

INSERT INTO DocumentEvent (DocumentID, EventID, SequenceNumber)
VALUES (p_document_id, p_event_id, p_sequence_number)
ON CONFLICT (DocumentID, EventID) DO NOTHING;
END;
$$;

```

-- 7. Процедура поручения мероприятия сотруднику

```

CREATE OR REPLACE PROCEDURE AssignEventToEmployee(
    p_event_id INT,
    p_employee_id INT,
    p_assignment_date DATE,
    p_priority VARCHAR(50)
)
LANGUAGE plpgsql
AS
$$
BEGIN
    INSERT INTO EventAssignment (EventID, EmployeeID, AssignmentDate, Priority)
    VALUES (p_event_id, p_employee_id, p_assignment_date, p_priority)
    ON CONFLICT (EventID, EmployeeID) DO NOTHING;
END;
$$;

```

-- 8. Функция для получения списка невыполненных мероприятий на заданную дату

```

CREATE OR REPLACE FUNCTION GetUncompletedEventsByDate(
    p_target_date DATE
)
RETURNS TABLE (
    DocumentNumber VARCHAR(50),
    EventName VARCHAR(200),
    Deadline DATE,
    CompletionDate DATE,
    IsCompleted BOOLEAN,
    CorrespondentFullName VARCHAR(100),
    CorrespondentPosition VARCHAR(100),
    DepartmentName VARCHAR(100),
    DocumentReleaseDate DATE --
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
    SELECT

```



```

    d.Number as DocumentNumber,
    e.Name as EventName,
    e.Deadline,
    e.CompletionDate,
    e.IsCompleted,
    c.FullName as CorrespondentFullName,
    c.Position as CorrespondentPosition,
    dep.Name as DepartmentName,
    d.ReleaseDate as DocumentReleaseDate
FROM Event e
INNER JOIN DocumentEvent de ON e.ID = de.EventID
INNER JOIN Document d ON de.DocumentID = d.ID
INNER JOIN DocumentCorrespondent dc ON d.ID = dc.DocumentID
INNER JOIN Correspondent c ON dc.CorrespondentID = c.ID
INNER JOIN CorrespondentDepartment cd ON c.ID = cd.CorrespondentID
INNER JOIN Department dep ON cd.DepartmentID = dep.ID
INNER JOIN DocumentType dt ON d.DocumentTypeID = dt.ID
WHERE e.Deadline <= p_target_date
      AND e.IsCompleted = FALSE
ORDER BY e.Deadline, d.Number;
END;
$$;

```

-- 9. Функция для получения списка мероприятий за период

```

CREATE OR REPLACE FUNCTION GetEventsByPeriod(

```

```

    p_start_date DATE,

```

```

    p_end_date DATE

```

```

)

```

```

RETURNS TABLE (

```

```

    EventName VARCHAR(200),

```

```

    Deadline DATE,

```

```

    CompletionDate DATE,

```

```

    IsCompleted BOOLEAN,

```

```

    DocumentNumber VARCHAR(50),

```

```

    DocumentTypeName VARCHAR(50)

```

```

)

```

```

LANGUAGE plpgsql

```

```

AS

```

```

$$

```

```

BEGIN

```

```

    RETURN QUERY

```

```

    SELECT

```

```

        e.Name as EventName,

```

```

    e.Deadline,
    e.CompletionDate,
    e.IsCompleted,
    d.Number as DocumentNumber,
    dt.TypeName as DocumentTypeName
FROM Event e
INNER JOIN DocumentEvent de ON e.ID = de.EventID
INNER JOIN Document d ON de.DocumentID = d.ID
INNER JOIN DocumentType dt ON d.DocumentTypeID = dt.ID
WHERE e.Deadline BETWEEN p_start_date AND p_end_date
ORDER BY e.Deadline, d.Number;
END;
$$;

```

-- 10. Функция для получения документов по типу на текущую дату

```

CREATE OR REPLACE FUNCTION GetDocumentsByTypeCurrentDate(
    p_document_type_name VARCHAR(50)
)
RETURNS TABLE (
    CompanyName VARCHAR(100),
    CurrentDate DATE,
    DocumentTypeName VARCHAR(50),
    DocumentNumber VARCHAR(50),
    CreationDate DATE,
    IsCompleted BOOLEAN
)
LANGUAGE plpgsql
AS
$$
BEGIN
    RETURN QUERY
    SELECT
        comp.Name as CompanyName,
        CURRENT_DATE as CurrentDate,
        dt.TypeName as DocumentTypeName,
        d.Number as DocumentNumber,
        d.CreationDate,
        EXISTS (
            SELECT 1
            FROM DocumentEvent de
            INNER JOIN Event e ON de.EventID = e.ID
            WHERE de.DocumentID = d.ID AND e.IsCompleted = FALSE
        ) as IsCompleted

```

```

FROM Document d
INNER JOIN DocumentType dt ON d.DocumentTypeID = dt.ID
CROSS JOIN Company comp
WHERE dt.TypeName = p_document_type_name
AND d.CreationDate <= CURRENT_DATE
ORDER BY d.CreationDate DESC, d.Number;
END;
$$;

```

```

-- 11. Триггер для автоматического обновления статуса выполнения мероприятия
CREATE OR REPLACE FUNCTION UpdateEventCompletionStatus()
RETURNS TRIGGER
LANGUAGE plpgsql
AS
$$
BEGIN
    IF NEW.CompletionDate IS NOT NULL AND OLD.CompletionDate IS NULL THEN
        NEW.IsCompleted := TRUE;
    ELSIF NEW.CompletionDate IS NULL AND OLD.CompletionDate IS NOT NULL THEN
        NEW.IsCompleted := FALSE;
    END IF;

    RETURN NEW;
END;
$$;

```

```

-- 12. Триггер для автоматического обновления статуса выполнения мероприятия
CREATE TRIGGER EventCompletionStatusTrigger
BEFORE UPDATE ON Event
FOR EACH ROW
EXECUTE FUNCTION UpdateEventCompletionStatus();

```

```

-- 13. Триггер для проверки дат документа
CREATE OR REPLACE FUNCTION CheckDocumentDates()
RETURNS TRIGGER
LANGUAGE plpgsql
AS
$$
BEGIN
    IF NEW.ReleaseDate < NEW.CreationDate THEN
        RAISE EXCEPTION 'Дата выхода документа не может быть раньше даты создания';
    END IF;

```

```

    RETURN NEW;
END;
$$;

-- 14. Триггер для проверки дат документа
CREATE TRIGGER DocumentDatesCheckTrigger
    BEFORE INSERT OR UPDATE ON Document
    FOR EACH ROW
    EXECUTE FUNCTION CheckDocumentDates();

-- 15. Процедура для массового добавления мероприятий к документу
CREATE OR REPLACE PROCEDURE AddEventsArrayToDocument(
    p_document_id INT,
    p_event_ids INT[],
    p_sequence_numbers INT[]
)
LANGUAGE plpgsql
AS
$$
DECLARE
    i INT;
    event_id INT;
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Document WHERE ID = p_document_id) THEN
        RAISE EXCEPTION 'Документ с ID % не существует', p_document_id;
    END IF;

    FOR i IN 1..array_length(p_event_ids, 1)
    LOOP
        event_id := p_event_ids[i];

        IF NOT EXISTS (SELECT 1 FROM Event WHERE ID = event_id) THEN
            RAISE NOTICE 'Мероприятие с ID % не существует, пропускаем', event_id;
            CONTINUE;
        END IF;

        INSERT INTO DocumentEvent (DocumentID, EventID, SequenceNumber)
        VALUES (p_document_id, event_id, p_sequence_numbers[i])
        ON CONFLICT (DocumentID, EventID) DO NOTHING;
    END LOOP;
END;
$$;

```

-- 16. Функция для получения всех исполнителей мероприятия

```
CREATE OR REPLACE FUNCTION GetEventExecutors(
    p_event_id INT
)
RETURNS TABLE (
    EmployeeID INT,
    FullName VARCHAR(100),
    EmployeePosition VARCHAR(100),
    AssignmentDate DATE,
    Priority VARCHAR(50)
)
LANGUAGE plpgsql
AS
$$
BEGIN
    RETURN QUERY
    SELECT
        e.ID as EmployeeID,
        e.FullName,
        e.Position as EmployeePosition,
        ea.AssignmentDate,
        ea.Priority
    FROM Employee e
    INNER JOIN EventAssignment ea ON e.ID = ea.EmployeeID
    WHERE ea.EventID = p_event_id
    ORDER BY ea.Priority, ea.AssignmentDate;
END;
$$;
```

-- 17. Функция создания сотрудника

```
CREATE OR REPLACE FUNCTION CreateEmployee(
    p_full_name VARCHAR(100),
    p_position VARCHAR(100)
)
RETURNS INT
LANGUAGE plpgsql
AS
$$
DECLARE
    new_id INT;
BEGIN
    INSERT INTO Employee (FullName, Position)
    VALUES (p_full_name, p_position)
```

```

RETURNING ID INTO new_id;

RETURN new_id;
END;
$$;

-- 18. Процедура редактирования сотрудника
CREATE OR REPLACE PROCEDURE UpdateEmployee(
    p_employee_id INT,
    p_full_name VARCHAR(100),
    p_position VARCHAR(100)
)
LANGUAGE plpgsql
AS
$$
BEGIN
    UPDATE Employee
    SET FullName = p_full_name,
        Position = p_position
    WHERE ID = p_employee_id;
END;
$$;

-- 19. Функция создания корреспондента
CREATE OR REPLACE FUNCTION CreateCorrespondent(
    p_full_name VARCHAR(100),
    p_position VARCHAR(100)
)
RETURNS INT
LANGUAGE plpgsql
AS
$$
DECLARE
    new_id INT;
BEGIN
    INSERT INTO Correspondent (FullName, Position)
    VALUES (p_full_name, p_position)
    RETURNING ID INTO new_id;

    RETURN new_id;
END;
$$;

```

-- 20. Процедура назначения сотрудника в подразделение

```
CREATE OR REPLACE PROCEDURE AssignEmployeeToDepartment(  
    p_employee_id INT,  
    p_department_id INT,  
    p_start_date DATE,  
    p_status VARCHAR(50)  
)  
LANGUAGE plpgsql  
AS  
$$  
BEGIN  
    INSERT INTO EmployeeDepartment (EmployeeID, DepartmentID, StartDate, Status)  
    VALUES (p_employee_id, p_department_id, p_start_date, p_status)  
    ON CONFLICT (EmployeeID, DepartmentID) DO UPDATE  
    SET StartDate = EXCLUDED.StartDate,  
        Status = EXCLUDED.Status;  
END;  
$$;
```

-- 21. Функция создания типа документа

```
CREATE OR REPLACE FUNCTION CreateDocumentType(  
    p_type_name VARCHAR(50)  
)  
RETURNS INT  
LANGUAGE plpgsql  
AS  
$$  
DECLARE  
    new_id INT;  
BEGIN  
    INSERT INTO DocumentType (TypeName)  
    VALUES (p_type_name)  
    ON CONFLICT (TypeName) DO NOTHING  
    RETURNING ID INTO new_id;  
  
    RETURN new_id;  
END;  
$$;
```

-- 22. Процедура редактирования типа документа

```
CREATE OR REPLACE PROCEDURE UpdateDocumentType(  
    p_type_id INT,  
    p_new_type_name VARCHAR(50)
```

```

)
LANGUAGE plpgsql
AS
$$
BEGIN
    UPDATE DocumentType
    SET TypeName = p_new_type_name
    WHERE ID = p_type_id;
END;
$$;

```

-- 23. Процедура удаления типа документа

-- Необходимо проверить, что нет документов этого типа, или обработать их.

```

CREATE OR REPLACE PROCEDURE DeleteDocumentType(
    p_type_id INT
)
LANGUAGE plpgsql
AS
$$
BEGIN
    IF EXISTS (SELECT 1 FROM Document WHERE DocumentTypeID = p_type_id) THEN
        RAISE EXCEPTION 'Невозможно удалить тип документа с ID %: существуют
документы этого типа.', p_type_id;
    END IF;
    DELETE FROM DocumentType WHERE ID = p_type_id;
END;
$$;

```


Интерфейс приложения: Основные окна с описанием

Учёт приказов и распоряжений

Документы	Мероприятия	Корреспонденты	Сотрудники	Отчёты	Справочники		
Добавить		Редактировать		Удалить		Обновить	
ID	Номер	Дата создания	Дата выхода	Содержание	Тип	Подписавший (должность)	Подписавший (ФИО)
1 11	P-2025-001	01.12.2025	02.12.2025	Утвердить план мероприятия...	Приказ	Генеральный директор	Петров П.П.
2 12	P-2025-001	05.12.2025	05.12.2025	Назначить ответственных за...	Распоряжение	Главный инженер	Иванов И.И.
3 13	У-2025-001	10.12.2025	10.12.2025	О введении в действие ново...	Указание	Начальник отдела разработки	Сидоров С.С.

Рисунок 11. Список данных во вкладке “Документы”

Добавить/Редактировать документ

Номер:

Дата создания: 17.12.25

Дата выхода: 17.12.25

Содержание:

Тип:

Приказ

Должность подписавшего:

ФИО подписавшего:

Сохранить

Отмена

Рисунок 12. Добавление/редактирование данных во вкладке “Документы”

Учёт приказов и распоряжений

Документы

Мероприятия

Корреспонденты

Сотрудники

Отчёты

Справочники

Добавить мероприятие

Редактировать

Удалить

Обновить

Привязать к документу

Назначить исполнителя

	ID	Название	Дедлайн	Дата выполнения	Выполнено
1 23		Назначить ответственных за оборудование	06.12.2025	06.12.2025	true
2 22		Обновить инструкции по охране труда	10.12.2025	10.12.2025	true
3 24		Ознакомить персонал с новым регламентом	11.12.2025		false
4 19		Проверить пожарные извещатели в корпусе А	15.12.2025	14.12.2025	true
5 20		Провести инструктаж по пожарной безопасности	18.12.2025		false
6 21		Подготовить отчёт по проверке охраны труда	20.12.2025		false

Рисунок 13. Список данных во вкладке “Мероприятия”

Добавить/Редактировать мероприятие

Название:

Дедлайн: 17.12.25

Дата выполнения: 1.01.00

Выполнено: Нет

Сохранить

Отмена

Рисунок 14. Добавление/редактирование данных во вкладке “Мероприятия”

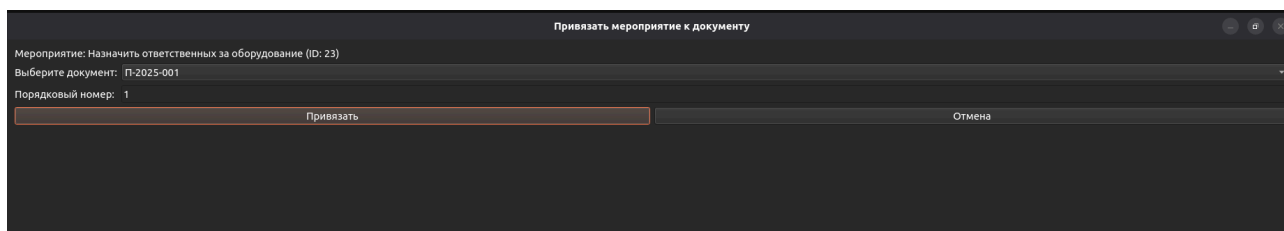


Рисунок 15. Привязка мероприятия к документу во вкладке “Мероприятия”

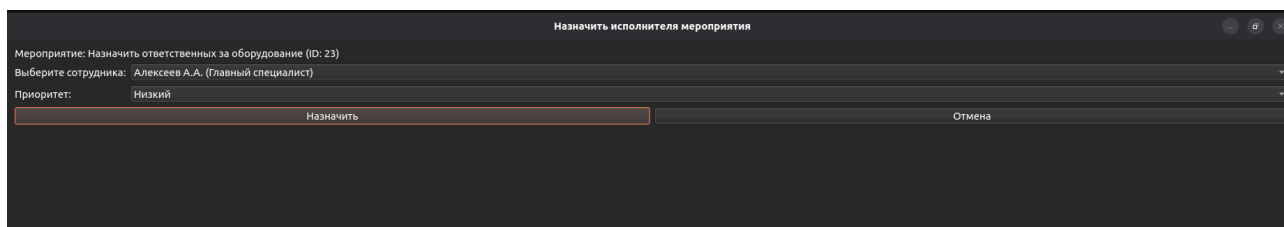


Рисунок 16. Назначение исполнителя мероприятия во вкладке “Мероприятия”

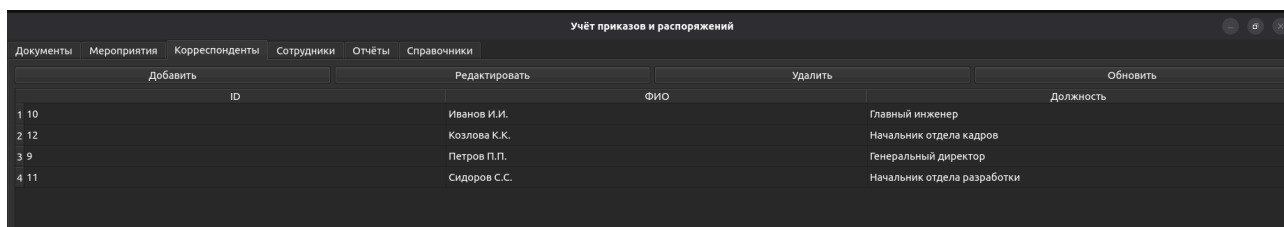


Рисунок 17. Список данных во вкладке “Корреспонденты”

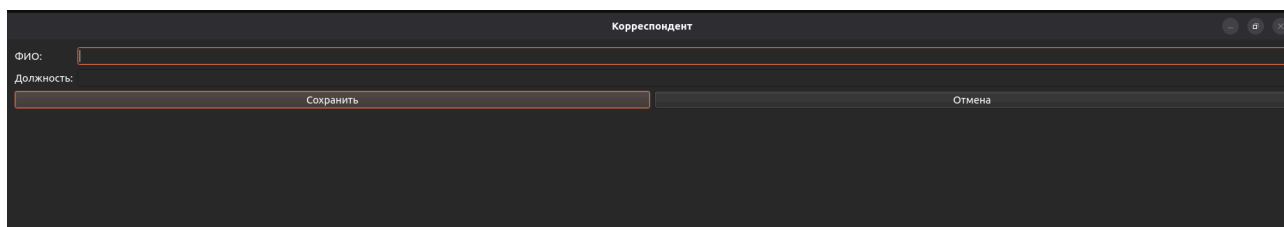


Рисунок 18. Добавление/редактирование данных во вкладке “Корреспонденты”

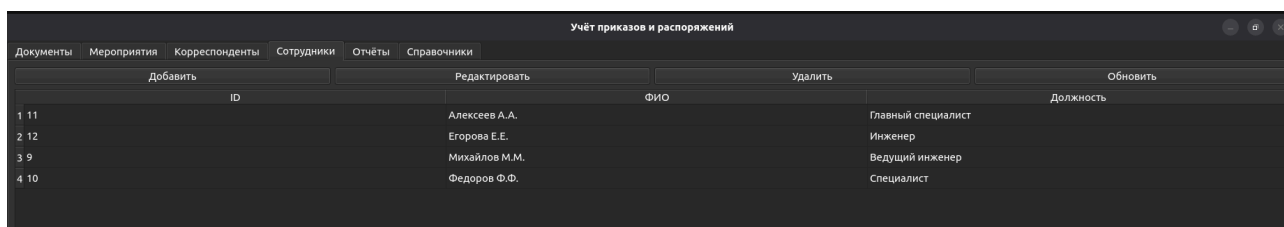


Рисунок 19. Список данных во вкладке “Сотрудники”

Рисунок 20. Добавление/редактирование данных во вкладке “Сотрудники”

ID	Название типа
1 11	Приказ
2 12	Распоряже...
3 13	Указание

Рисунок 21. Список данных во вкладке “Справочники”

Рисунок 22. Добавление/редактирование данных во вкладке “Справочники”

Номер документа	Название мероприятия	Дедлайн	Дата выполнения	Выполнено	ФИО корреспондента	Должность корреспондента	Название подразделения	Дата выхода документа
1 У-2025-001	Ознакомить персонал с ...	11.12.2025		False	Сидоров С.С.	Начальник отдела ...	Отдел разработки	10.12.2025

Рисунок 23. Показать невыполненные мероприятия на дату

Название мероприятия	Дедлайн	Дата выполнения	Выполнено	Номер документа	Указание	Тип документа
1 Ознакомить персонал с новым ...	11.12.2025		False	У-2025-001	Указание	Приказ
2 Проверить пожарные извещатели в ...	15.12.2025	14.12.2025	True	П-2025-001	Приказ	

Рисунок 24. Показать мероприятия на период

Название предприятия	Текущая дата	Тип документа	Номер документа	Дата документа	Пометка о выполнении
1 ООО "Рига и Колыта"	17.12.2025	Приказ	П-2025-001	01.12.2025	True

Рисунок 25. Показать документы заданного типа

Инструментальные средства, которые использованы при разработке

СУБД: PostgreSQL

Язык программирования: Python

Библиотека для создания оконного приложения: PyQt6